



Carrera de Ciencia de Datos
e Inteligencia Artificial
FACULTAD DE INGENIERÍA

Principios de modelado y validación de redes neuronales artificiales

Epochs (épocas-ciclo)

Es el recorrido completo con la muestra de entrenamiento, de manera que cada entrada x se haya procesado una vez.

Un epoch representa el número de iteraciones de entrenamiento, $N/\text{tamaño del lote}$, donde N es la cantidad total de ejemplos.

Si el conjunto de datos consta de 1,000 muestras de entrenamiento y el tamaño del lote es de 50 (minibatches), una sola época tendrá 20 iteraciones:

Así también, para el mismo caso, si se usa el lote completo, las 1000 muestras se procesan en una sola pasada, es decir el epoch corresponderá a una iteración por toda la muestra de entrenamiento.

En cambio si se trata de un entrenamiento estocástico, y el lote es igual a 1, el número de iteraciones es de 1000.

Lote pequeño $\rightarrow$ más iteraciones Lote grande $\rightarrow$ menos iteraciones

Lote = todo el dataset $\rightarrow$ 1 iteración

Sobreajuste

Creación de un modelo que coincide de tal manera con los datos de entrenamiento que no puede realizar predicciones correctas con datos nuevos.

La regularización puede reducir el sobreajuste. Aumentar la tasa de regularización reduce el sobreajuste, pero puede disminuir la capacidad predictiva del modelo.

Por el contrario, reducir u omitir la tasa de regularización aumenta el sobreajuste.

Entrenar el modelo con un conjunto de datos de entrenamiento grande y diverso también puede reducir el sobreajuste.

Subajuste

Producir un **modelo con poca capacidad predictiva** porque el modelo no ha capturado por completo la complejidad de los datos de entrenamiento. El subajuste puede estar causado por varios problemas, como los siguientes:

- Entrenamiento con el conjunto incorrecto de **atributos**
- Entrenamiento con pocos **ciclos** o con una **tasa de aprendizaje** demasiado baja
- Entrenamiento con una **tasa de regularización** demasiado alta
- Establecer muy pocas **capas ocultas** en una red neuronal profunda

Pérdida en el entrenamiento

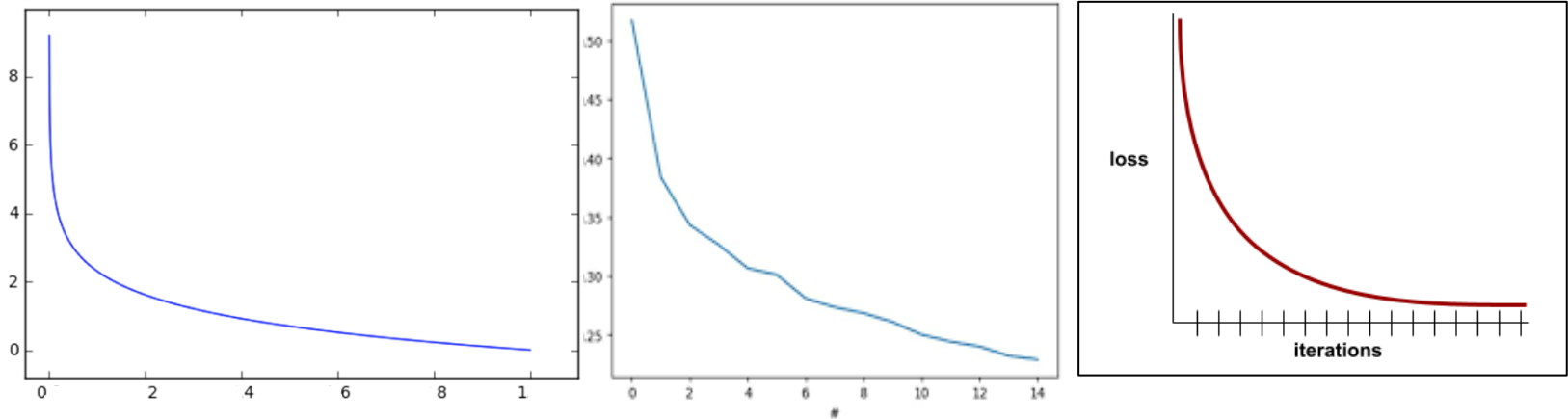
Es una métrica que representa la pérdida de un modelo durante una iteración de entrenamiento en particular.

Si por ejemplo, la función de pérdida es el error cuadrático medio, y su valor a la décima iteración es de 2.2, se esperaría su decrecimiento, llegando a la iteración número 100 a tener un valor de 1.9.

Para ilustrarlo se suele usar la curva de pérdida de entrenamiento, representada como con respecto a la cantidad de iteraciones. La curva puede ser interpretada con lo siguiente:

- Una pendiente descendente implica que el modelo está mejorando.
- Una pendiente ascendente implica que el modelo está empeorando.
- Una pendiente plana implica que el modelo alcanzó la convergencia.

Pérdida en el entrenamiento (cont.)



Una pendiente descendente pronunciada durante las iteraciones iniciales, lo que implica una mejora rápida del modelo

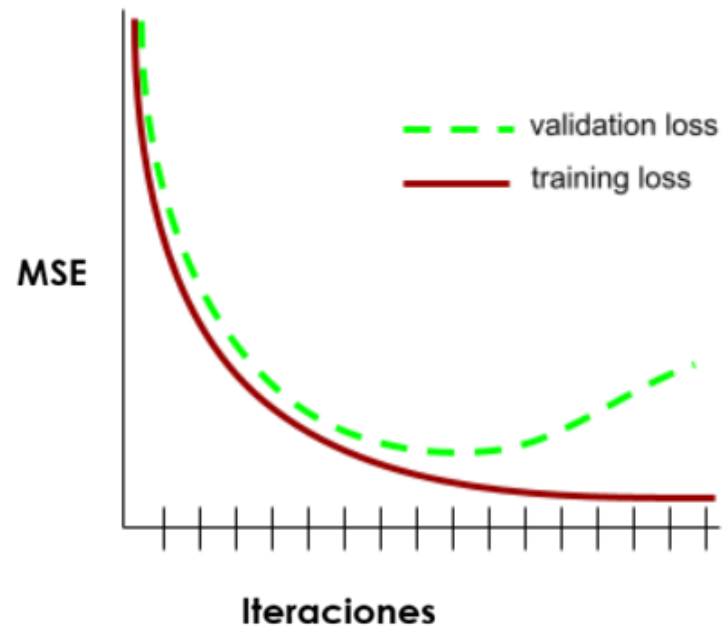
Una pendiente que se aplanan gradualmente (pero que sigue siendo descendente) hasta cerca del final del entrenamiento, lo que implica una mejora continua del modelo a un ritmo algo más lento que durante las iteraciones iniciales.

Una pendiente plana hacia el final del entrenamiento, lo que sugiere convergencia.

Generalización

Es la capacidad de un modelo para realizar predicciones correctas sobre datos nuevos (no usados durante el entrenamiento), correspondientes a la muestra de validación.

Un modelo que puede generalizar es lo opuesto a un modelo que tiene un sobreajuste.



En la figura se observa un caso en donde la curva de generalización sugiere **sobreajuste**, debido a que la pérdida de validación en última instancia se vuelve significativamente mayor que la pérdida de entrenamiento.

Hiperparámetros

Durante la configuración inicial de un modelo ML, el investigador define los **hiperparámetros**, cuya función es determinar cómo el modelo aprenderá de los datos, siendo los siguientes:

Aquellos que inciden directamente en la optimización y convergencia:

- Tasa de aprendizaje (Ejemplo: 0.01, 0.03), si es muy alta tiende a desestabilizar la dinámica de aprendizaje (divergencia); si es muy pequeña aprende de manera lenta.
- Algoritmo de aprendizaje , apoyado en la función de costo, para actualizar los pesos (Descenso del gradiente, Descenso del gradiente adaptable – AdaGrad, Adaptativo con momento- ADAM, entre otros).

Hiperparámetros (cont.)

Los hiperparámetros que definen la estructura de la red neuronal son los siguientes:

- Número de capas (cuántas capas ocultas tiene la red)
- Número de neuronas por capa
- Funciones de activación

Los hiperparámetros que controlan el aprendizaje son los siguientes:

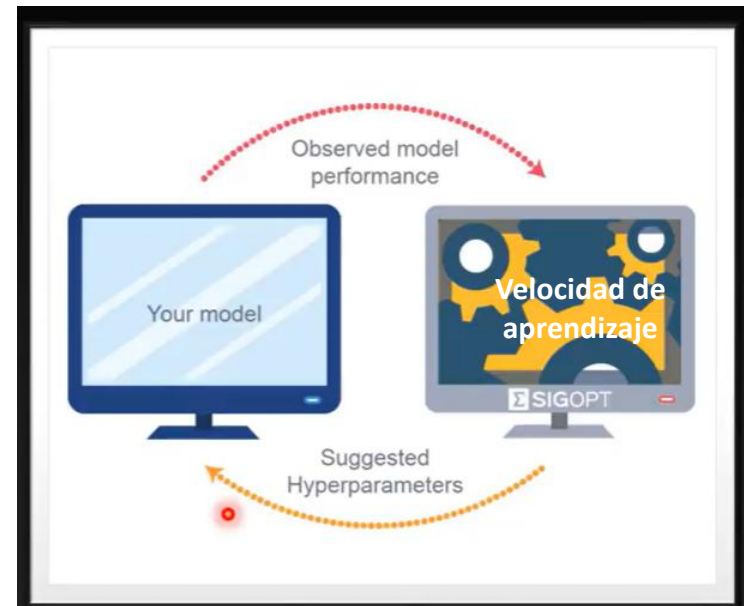
- Tamaño del Batch
- Número de épocas/iteraciones
- Buffer, pasos.

Parámetros

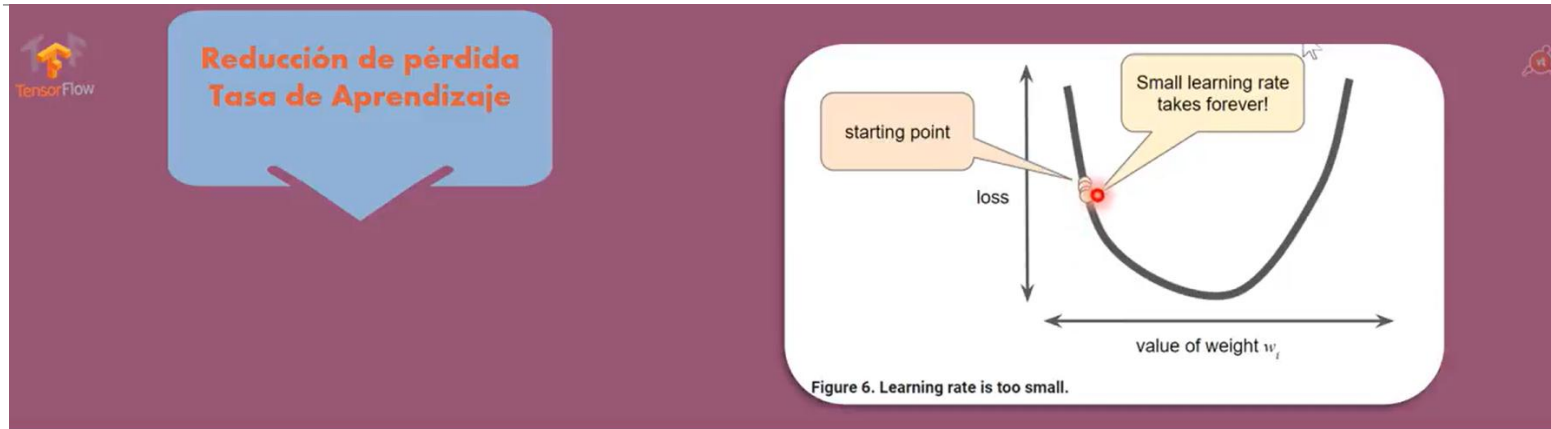
Los **parámetros** son elementos internos del modelo, se configuran y ajustan durante el proceso de aprendizaje, en la fase de entrenamiento, constituyen los **pesos** y el **sesgo (bias)**.

Los **parámetros** son definidos por el modelo.

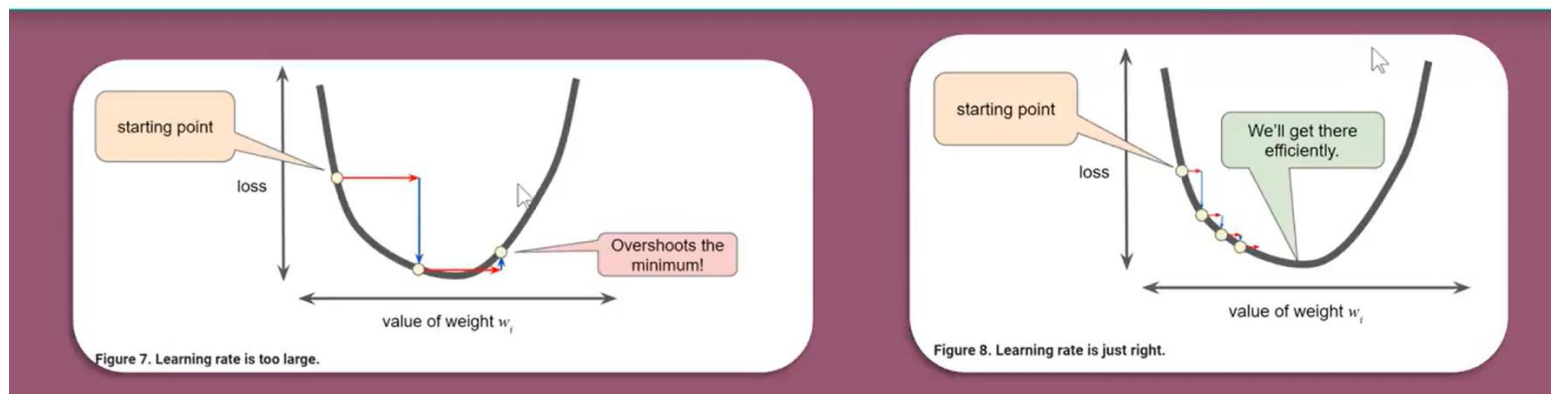
Los **hiperparámetros** se definen antes del proceso de entrenamiento y su ajuste es determinante para encontrar los parámetros. Son de nivel superior y no pueden ser ajustados durante el entrenamiento.



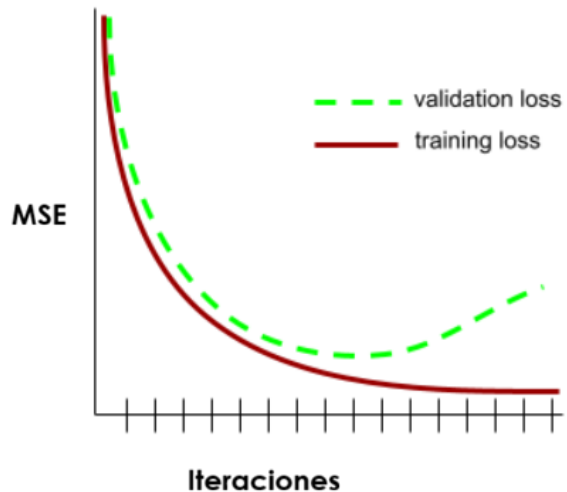
Tasa de aprendizaje



Nuevo peso = antiguo peso - tasa de aprendizaje * gradiente



Resultados de Función de costo, ajuste de hiperparámetros



¿Cuáles son los mejores valores para los hiperparámetros?



Matriz de confusión

Mecanismo para evaluar el desempeño del aprendizaje automático en problemas de clasificación.

La matriz de confusión muestra los porcentajes de acierto y desacierto, etiquetados como:

VERDADEROS POSITIVOS	VERDADEROS NEGATIVOS	ACIERTOS
FALSOS POSITIVOS	FALSOS NEGATIVOS	DESACIERTOS

Matriz de confusión binaria

					
		positivo	negativo		
		categoría real	categoría predicha	esto es un...	
ACIERTO				Verdadero <u>P</u> ositivo (VP)	
				Falso <u>N</u> egativo (FN)	
ACIERTO				Verdadero <u>N</u> egativo (VN)	
				Falso <u>P</u> ositivo (FP)	

		categorías reales	
		normal	anormal
categorías predichas	normal	89	1
	anormal	9	1

VP es positivo y le dijo positivo
 VN es negativo y le dijo negativo
 FP es positivo y le dijo negativo
 FN es negativo y le dijo positivo

Existen 97 normales, pero solo fueron clasificados como tal 89. Existen 2 anormales, y solo se hizo la clasificación correcta de 1.

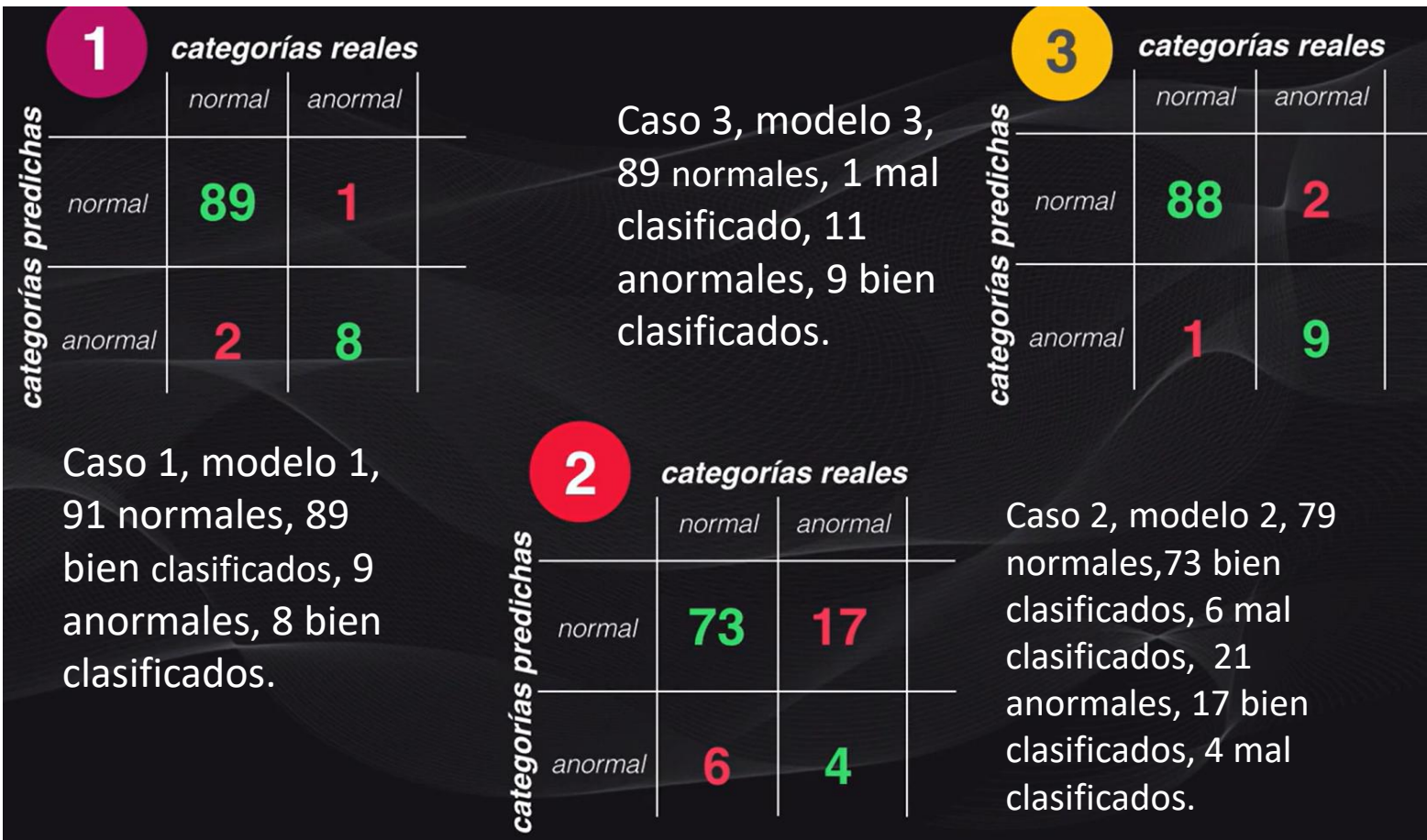
Matriz de confusión multiclase

		<i>categorías reales</i>				
		<i>normal</i>	<i>anormal tipo 1</i>	<i>anormal tipo 2</i>	<i>anormal tipo 3</i>	<i>anormal tipo 4</i>
<i>categorías predichas</i>	<i>normal</i>	53	2	1	0	4
	<i>anormal tipo 1</i>	1	7	2	0	0
	<i>anormal tipo 2</i>	1	0	8	0	1
	<i>anormal tipo 3</i>	0	0	0	10	0
	<i>anormal tipo 4</i>	1	2	1	2	4

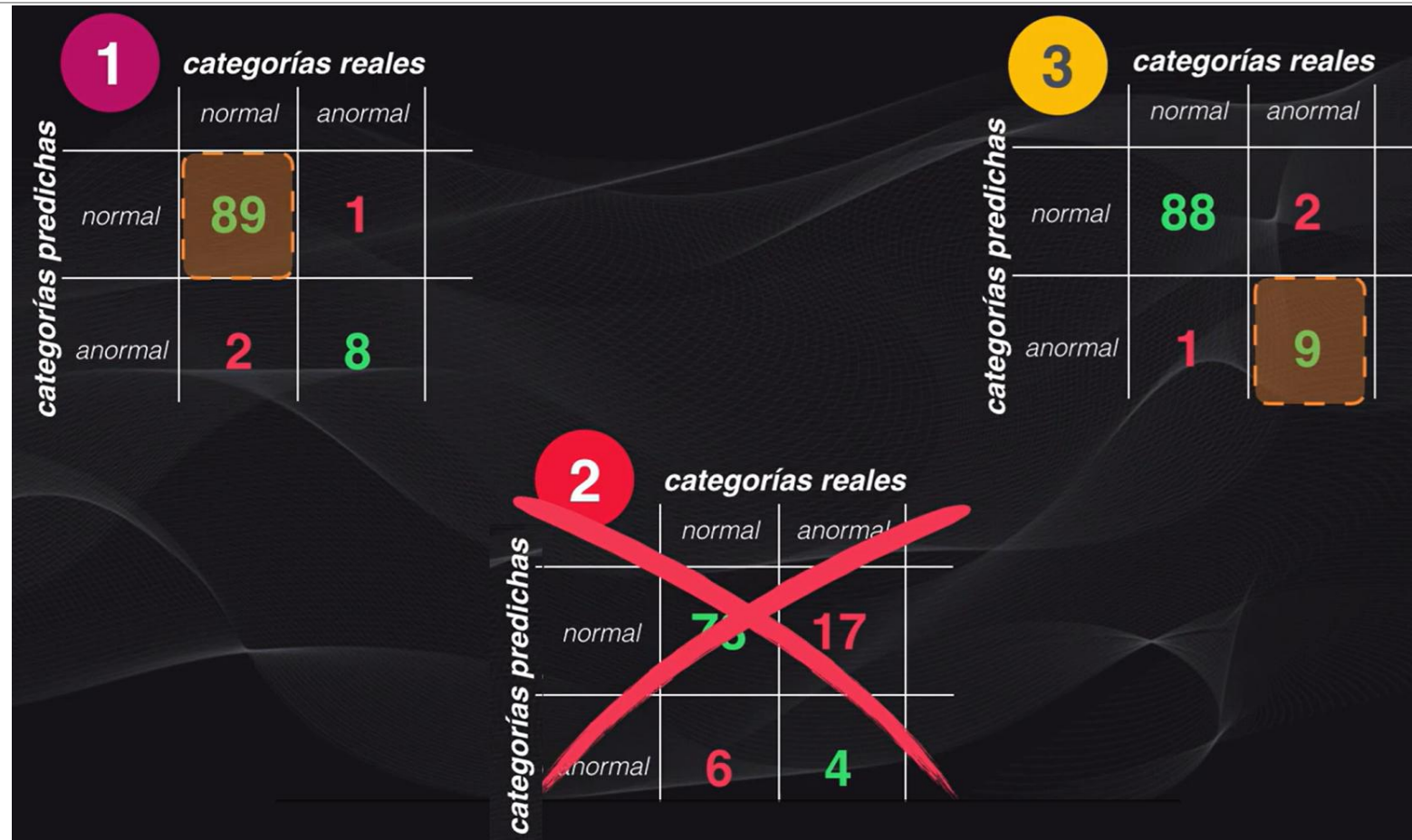
De la clase normal 53 aciertos, 3 desaciertos, De la clase anormal tipo 1 tuvo 4 desaciertos, de la clase anormal tipo 4 tuvo 5 desaciertos.

“Diagonal principal -> aciertos, otras celdas, desaciertos”

Comparación de resultados de la matriz de confusión



Comparación de resultados de la matriz de confusión



Métricas calculadas a partir de la matriz de confusión

$$ACCURACY = \frac{\text{total de aciertos}}{\text{total de datos}}$$

No distingue categorías

$$PRECISION = \frac{TP}{TP + FP}$$

% datos clasificados como positivos que son realmente positivos

$$RECALL = \frac{TP}{TP + FN}$$

% positivos reales clasificados como positivos

$$F1 - SCORE = 2 \frac{PRECISION \cdot RECALL}{PRECISION + RECALL}$$

Media armónica entre precision y recall

Clases desbalanceadas

Clases desbalanceadas (o imbalanced classes) se refieren a un problema donde ciertas categorías tienen supremacía de datos con respecto a otra (s) clase (s).

Por ejemplo, en un caso de detección de Spam, se obtienen

- No es SPAM → 9,900 casos
- Es SPAM → 100 casos

Existe un desbalance fuerte, la clase mayoritaria es NO SPAM, frente a la clase minoritaria: SPAM.

Constituye un problema, debido a que el modelo estará detectando con alta exactitud casos regulares (NO SPAM), si se usa la métrica ACCURACY, puede llegar a ser engañosa.

Clases desbalanceadas (cont.)

Cuando existe desbalance, se deben usar las otras métricas (Precision, Recall (sensibilidad), F1-score)

Cuando existe desbalanceo se recomienda las siguientes opciones:

- Oversampling (aumentar la clase minoritaria) o Undersampling (reducir la mayoritaria)
- Aplicar ponderación de las clases, asignar mayor “peso” a la clase minoritaria durante el entrenamiento.
- Ajustar el umbral (cambiar el punto de decisión del modelo).
- Aplicar algoritmos robustos (Random Forest, XGBoost, etc.), con manejo de pesos.

Aplicación - Experimentación

1. Hacer uso de la librería SCIKIT LEARN para evaluar el desempeño de un modelo de clasificación binaria, por medio de la matriz de confusión, calcular accuracy, precision, recall y f1-score.

Definir arreglos con valores reales (y_true) y valores predichos (y_pred) con valores binarios (0,1)

Importar matriz de confusión de la librería (from sklearn.metrics import confusion_matrix)

Calcular la matriz de confusión confusion_matrix(y_true, y_pred)

Graficar la matriz de confusión

```
from sklearn.metrics import ConfusionMatrixDisplay
```

```
ConfusionMatrixDisplay.from_predictions(y_true, y_pred);
```

Importar métricas y visualizar resultados

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

```
print ("Exactitud:", accuracy_score(y_true, y_pred))...
```

Generar un reporte de clasificación e interpretar los resultados obtenidos

```
from sklearn.metrics import classification_report
```

Aplicación - Experimentación

2. Evaluar el desempeño de un modelo de **clasificación multiclase**, por medio de la matriz de confusión, F1SCORE, RECALL.

Definir arreglos con valores reales (y_true) y valores predichos (y_pred) con los valores de las clases (0,1,2,3...)

Continuar el proceso similar al de la práctica anterior.

Material basado en la bibliografía
declarada en el sílabo.
